

ROS2 Spring Workshop

Lab 10: The Farm Manager

(Safety Systems, Overrides & Ackermann Mission Execution)

Project Context: The Final Integration

In Lab 7 you generated a waypoint path and followed it with a P-controller in simulation. In Lab 9 you drove your physical Ackermann robot to a single real-world goal using the ceiling camera. In Lab 6 you built a weed detector.

Today you wire them together. Your robot will follow a multi-waypoint mission like the one from Lab 7, driven by real-world GPS from Lab 9, while a human operator retains absolute override authority through a keyboard-based deadman switch. This is the same control architecture used on autonomous tractors, surgical robots, and self-driving cars.

Objective

You will build two nodes that work together:

1. The **Farm Manager**, a safety-critical arbiter that decides whether manual or autonomous commands reach the motors, and enforces a keyboard deadman switch.
2. The **Mission Runner**, an upgraded Lab 9 controller that follows a multi-waypoint path using **Pure Pursuit**, the right steering algorithm for an Ackermann vehicle.

You will learn:

- **The arbiter pattern:** how multiple control sources get resolved into a single safe command.
- **The deadman switch:** the non-negotiable safety primitive in industrial robotics.
- **Pure Pursuit:** the Ackermann-native path-following algorithm used in every real autonomous car and tractor.
- **Single-arc headland turns:** using parametric arc geometry (from Lab 7) to transition between rows on a car-like robot.

Critical Rule: Command Authority

Only the Farm Manager may publish to `/cmd_vel`.

No other node is allowed to write directly to the motors.

If more than one node publishes to `/cmd_vel`, your safety system is broken.

Check this with:

```
1 ros2 topic info /cmd_vel
```

Pre-Lab Concept Primer

1. The Deadman Switch (Non-Negotiable)

Autonomous machines must never be fully unattended. A human operator holds a physical input to grant the robot permission to move. If they slip, faint, or get distracted, the machine stops within milliseconds.

Today your deadman is the **spacebar on your laptop keyboard**. **Held** → robot permitted to move. **Released** → zero command, no exceptions. The autopilot doesn't get a vote. The navigation algorithm doesn't get a vote. If the spacebar is not held, the published `cmd_vel` must be (0, 0).

This rule must be so obvious in the code that you cannot accidentally violate it.

Why the Keyboard?

In a real industrial setting, deadmen are dedicated physical buttons wired straight into the safety circuit. We're emulating that pattern with the laptop's spacebar.

A keyboard deadman is documented (you write the code), portable (works on every laptop in the room), and trivially easy to test. Press the key, watch the log, release the key, watch the log. The pedagogical point isn't the hardware; it's the pattern: a continuous hold-to-enable input that fails safe the instant it's released. Real safety systems use the same principle, just with sturdier buttons and certified wiring.

2. Why Ackermann Changes Everything

Your Lab 9 controller used simple "rotate toward goal, drive forward" logic. That works when the goal is in front of you. It fails on Ackermann vehicles when:

- The waypoint is *behind* you. Your robot cannot turn in place. It must drive forward in an arc.
- You're transitioning between parallel rows. A 180° direction change requires a **headland turn**, not a pivot.
- You're following a curved path. Constant goal-pointing produces oscillation.

Pure Pursuit solves all three. Instead of pointing at the goal, you pick a *look-ahead point* on the path some distance ahead and compute the *curvature* (1 / turning radius) needed to reach it. This maps directly onto Ackermann steering.

3. Your Robot's Geometry

Hiwonder Pi Ackermann:

- Wheelbase: ≈ 0.15 m
- Minimum turning radius: ≈ 0.35 m (full lock)
- Minimum turning *diameter*: ≈ 0.70 m

This means if two parallel rows are spaced more than 0.70 m apart, you can transition between them with a **single 180° arc**. If they're closer than 0.70 m, a single arc overshoots and you need a multi-phase manoeuvre (stretch goal).

For your 3 m × 3 m arena with 1 m row spacing, a single arc works cleanly.

Part 1: The Arbiter Pattern

In Lab 9, your controller published directly to `/cmd_vel`. That's fine for a single controller, but dangerous when a human also needs to drive. Today we insert a **Farm Manager** between all controllers and the motors:

This is the same pattern used in autonomous cars ("human override always wins"), surgical robots

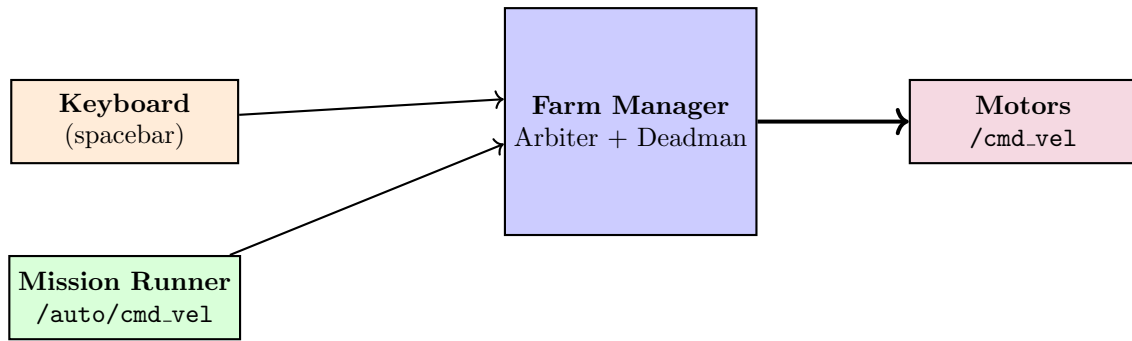


Figure 1: Only the Farm Manager writes to `/cmd_vel`. All other sources publish *intent* on prefixed topics; the Farm Manager decides what actually reaches the motors.

(“surgeon can disable tool motion”), and factory co-bots (“e-stop disables autonomous motion”). **The Mission Runner never writes directly to the motors. It only publishes intent to the Farm Manager.**

Part 2: Package Setup

Install the keyboard library

The deadman listener uses `pynput` to capture keyboard events with proper key-down and key-up detection. In your laptop’s ROS container:

```

1 sudo apt update
2 sudo apt install python3-pip # if pip3 is not installed
3 pip3 install pynput

```

Prerequisite

This lab uses lab9’s `fake_god_node` for bench testing. Make sure lab9 is copied into `src/` and builds cleanly before starting Part 6.

Create the package

```

1 cd /workspaces/agtech_ros2/src
2 ros2 pkg create --build-type ament_python lab10
3 mkdir -p lab10/lab10 lab10/launch

```

You will create two Python files in `lab10/lab10/`. These files are in the lab resource folder, it may require a fresh git pull:

- `farm_manager.py`, the arbiter with the keyboard deadman (Part 3)
- `mission_runner.py`, the Pure Pursuit mission runner (Part 5)

Add these to `setup.py`:

```

1 'console_scripts': [
2     'farm_manager = lab10.farm_manager:main',
3     'mission_runner = lab10.mission_runner:main',
4 ],

```

Part 3: The Farm Manager (Safety Arbiter)

The Farm Manager has three jobs:

1. Listen to the spacebar. Track whether it’s currently held.
2. Listen to `/auto/cmd_vel` from the Mission Runner.

3. Publish to `/cmd_vel` at a fixed rate, with the deadman check as the first thing it does.

Important: What `angular.z` Means

On this robot, `angular.z` is not a spin command.

It represents a desired yaw rate. The motor driver converts this into a steering angle using Ackermann geometry.

This has important consequences:

- The robot cannot turn in place
- Turning requires forward motion
- Very low forward speed can produce large steering angles

Create `farm_manager.py`:

```
1  #!/usr/bin/env python3
2  """
3  Farm Manager - the safety arbiter.
4
5  The keyboard's spacebar is the deadman switch. Hold it to enable motion,
6  release it to stop. The autonomous controller (Mission Runner) publishes
7  to /auto/cmd_vel, but those commands only reach /cmd_vel while the
8  spacebar is held AND the auto stream is fresh.
9
10 SAFETY: if the spacebar is not held, /cmd_vel is zero. Period.
11 SAFETY: if /auto/cmd_vel has gone stale, /cmd_vel is zero. Period.
12 """
13 import threading
14 import rclpy
15 from rclpy.node import Node
16 from geometry_msgs.msg import Twist
17 from pynput import keyboard
18
19
20 # --- ACKERMANN LIMITS ---
21 MAX_LINEAR = 0.25 # m/s, forward speed cap (passthrough clamp)
22 MAX_ANGULAR = 1.0 # rad/s, yaw rate cap
23
24 # --- STALENESS LIMIT ---
25 # If we haven't heard from the Mission Runner in this long, treat the
26 # autonomous command as dead and publish zero. Half a second is plenty
27 # of slack at the Mission Runner's 20 Hz publish rate.
28 AUTO_CMD_TIMEOUT = 0.5 # seconds
29
30
31 def clamp(x, lo, hi):
32     return max(lo, min(hi, x))
33
34
35 class FarmManager(Node):
36     def __init__(self):
37         super().__init__('farm_manager')
38
39         self.create_subscription(
40             Twist, '/auto/cmd_vel', self.on_auto_cmd, 10)
41         self.pub_cmd = self.create_publisher(Twist, '/cmd_vel', 10)
42
43         # --- Deadman state, written by the keyboard listener thread,
44         # read by the ROS timer. A lock keeps the read atomic. ---
45         self._lock = threading.Lock()
46         self._deadman_held = False
47         self._last_auto_cmd = Twist()
48         self._last_auto_time = self.get_clock().now()
49
50         # --- Keyboard listener runs in its own thread. pynput needs this
51         # to capture keys without blocking the ROS event loop. ---
52         self._listener = keyboard.Listener(
53             on_press=self._on_key_press,
54             on_release=self._on_key_release,
55         )
56         self._listener.daemon = True
```

```

57     self._listener.start()
58
59     # --- Fixed-rate publish. The deadman is checked here, NOT in the
60     # keyboard callbacks. This guarantees the check runs even if no
61     # keyboard event fires for a while. ---
62     self.create_timer(0.05, self.publish_cmd)    # 20 Hz
63
64     self.get_logger().info(
65         "Farm Manager ONLINE. "
66         "HOLD SPACEBAR to enable motion. "
67         "Focus this terminal window."
68     )
69
70     # -----
71     # Keyboard callbacks (run in the pynput thread, not the ROS thread)
72     # -----
73     def _on_key_press(self, key):
74         if key == keyboard.Key.space:
75             with self._lock:
76                 if not self._deadman_held:
77                     self.get_logger().info("DEADMAN engaged (spacebar held).")
78                     self._deadman_held = True
79
80     def _on_key_release(self, key):
81         if key == keyboard.Key.space:
82             with self._lock:
83                 if self._deadman_held:
84                     self.get_logger().info("DEADMAN released. Motors stopped.")
85                     self._deadman_held = False
86
87     # -----
88     # ROS callbacks
89     # -----
90     def on_auto_cmd(self, msg: Twist):
91         with self._lock:
92             self._last_auto_cmd = msg
93             self._last_auto_time = self.get_clock().now()
94
95     def publish_cmd(self):
96         cmd = Twist()
97
98         # =====
99         # SAFETY LAYER. FIRST, ABSOLUTE, NON-BYPASSABLE.
100        # =====
101        with self._lock:
102            held = self._deadman_held
103            auto = self._last_auto_cmd
104            auto_time = self._last_auto_time
105
106        # Gate 1: deadman must be held.
107        if not held:
108            self.pub_cmd.publish(cmd)    # zero
109            return
110
111        # Gate 2: auto command stream must be fresh. If the Mission
112        # Runner crashes mid-mission, we don't want to keep replaying
113        # its last command forever just because the student is still
114        # holding the spacebar.
115        age = (self.get_clock().now() - auto_time).nanoseconds / 1e9
116        if age > AUTO_CMD_TIMEOUT:
117            self.get_logger().warn(
118                f"Auto command stale ({age:.1f}s). Holding zero.",
119                throttle_duration_sec=2.0)
120            self.pub_cmd.publish(cmd)    # zero
121            return
122
123        # Both gates passed. Pass through the autonomous command, clamped.
124        cmd.linear.x = clamp(auto.linear.x, -MAX_LINEAR, MAX_LINEAR)
125        cmd.angular.z = clamp(auto.angular.z, -MAX_ANGULAR, MAX_ANGULAR)
126        self.pub_cmd.publish(cmd)
127
128
129 def main():

```

```

130 rclpy.init()
131 node = FarmManager()
132 try:
133     rclpy.spin(node)
134 except KeyboardInterrupt:
135     pass
136 finally:
137     # SAFETY: publish zero on shutdown. Always.
138     stop = Twist()
139     node.pub_cmd.publish(stop)
140     node.destroy_node()
141     rclpy.shutdown()
142
143
144 if __name__ == '__main__':
145     main()

```

Design choices worth noticing

The safety check runs on a timer, not just when keys are pressed. A keypress callback only fires when you actually press or release a key. If you stop touching the keyboard for a few seconds, no callback fires. That means the deadman check needs its own heartbeat (the 20 Hz timer) so the safety logic runs 20 times a second whether you're touching the keyboard or not.

The deadman check happens first. Look at `publish_cmd`: the very first thing it does is check whether the spacebar is held, and if not, it publishes zero and stops. Nothing later in the function can undo that. Safety logic should be impossible to accidentally bypass, so it goes at the top.

Two safety gates, not one. The spacebar protects against the human walking away. The auto-staleness check protects against the Mission Runner crashing. If the Mission Runner dies mid-mission while you're still holding the spacebar, without the staleness check the Farm Manager would happily keep republishing its last cached command forever. The half-second timeout catches that.

The keyboard listener runs separately from the ROS code. `pynput` (the keyboard library) wants to take over the program and block everything else while it waits for keys. ROS also wants to take over and block everything else while it processes messages. They cannot both do that at the same time, so we give the keyboard listener its own track to run on. The two pieces then run side by side without fighting.

Shared variables get protected. The keyboard side writes `_deadman_held` and `_last_auto_cmd`. The ROS timer side reads them. Whenever two parts of the program touch the same variable, you wrap the read and write in a `with self._lock`: block so they do not step on each other. It costs almost nothing and prevents weird bugs that are nearly impossible to debug later.

Critical: Terminal Focus

`pynput` captures keyboard events globally, not just from one window. **This means the spacebar works no matter which application is focused on your laptop**, including the terminal where Farm Manager is running, your browser, or VS Code.

Pros: you can monitor the robot in one window and have the deadman work from anywhere.

Cons: if you tab over to a chat app and hit space while typing, the robot will move. **Do not browse the web while the robot is autonomous.** Keep the laptop focused on the lab.

Part 4: Safety Test

Before writing the mission runner, prove the safety system works. This test is mandatory.

```

1 # Terminal 1: build and run the manager
2 cd /workspaces/agtech_ros2
3 colcon build --packages-select lab10
4 source install/setup.bash
5 ros2 run lab10 farm_manager
6
7 # Terminal 2: watch what the motors would see
8 ros2 topic echo /cmd_vel
9
10 # Terminal 3: fake an autonomous command stream

```

```

11 ros2 topic pub /auto/cmd_vel geometry_msgs/msg/Twist \
12 "linear: {x: 0.2}, angular: {z: 0.3}" --rate 10

```

Safety Test Checklist

Robot on a block, wheels free. Not on the floor. Check every one of these before moving on.

1. Spacebar **released**, with Terminal 3 streaming commands. `/cmd_vel` in Terminal 2 must stay zero.
2. Spacebar **held down**. `/cmd_vel` starts publishing the autonomous values (approximately 0.2 linear, 0.3 angular).
3. Release the spacebar *while values are streaming*. `/cmd_vel` must snap to zero within 50 ms.
4. Kill Terminal 3 (Ctrl+C), then hold spacebar. `/cmd_vel` should be zero within half a second (auto-staleness check kicks in).
5. Ctrl+C on the Farm Manager itself. The final published `/cmd_vel` should be zero.

If any of these fail, do not continue. The safety layer must work before you put a real robot on the floor.

Deadman Debug Check

Before testing motion, confirm that the keyboard listener is working.

Press the spacebar and watch Terminal 1.

You should see:

- “DEADMAN engaged” when pressed
- “DEADMAN released” when released

If you do not see these messages, the keyboard listener is not working.

Do not continue until this is fixed.

Part 5: The Mission Runner (Pure Pursuit)

Your Lab 9 controller drove to *one* goal. The Mission Runner drives through a *sequence* of goals using **Pure Pursuit**, the same algorithm used in real autonomous vehicles.

How Pure Pursuit works

Instead of pointing at the current waypoint, Pure Pursuit picks a **look-ahead point** on the path a fixed distance ahead of the robot, then commands the curvature needed to reach that point along an arc:

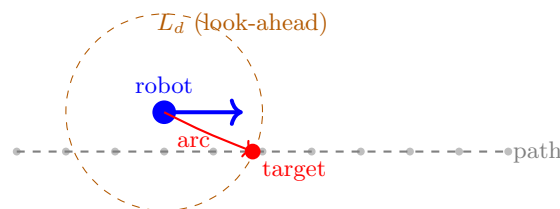


Figure 2: Pure Pursuit: find the closest breadcrumb at least L_d metres ahead, then curve toward it.

The math:

- α = angle from robot's heading to the look-ahead point
- curvature $\kappa = \frac{2 \sin(\alpha)}{L_d}$

- yaw rate to send to motors: $\omega = \kappa \cdot v$

Where v is forward speed. Multiplying curvature by speed gives the yaw rate, which is what `cmd_vel.angular.z` expects.

Look-ahead distance tuning:

- Too short: robot oscillates (oversteers)
- Too long: robot cuts corners
- Good starting value for your robot: $L_d = 0.3\text{ m}$

Breadcrumbs from Lab 7

You already know how to generate a breadcrumb path. You did it in Lab 7 with linear interpolation and parametric arcs. Today we reuse that code style. The Mission Runner generates a zigzag path with two rows joined by a single-arc headland turn:

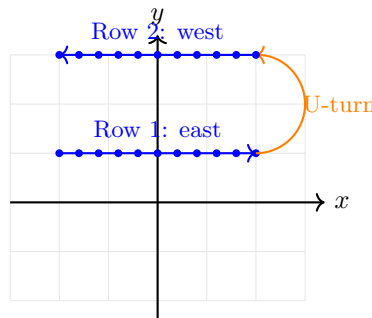


Figure 3: A two-row zigzag with a single-arc headland turn of radius 0.5 m (larger than the 0.35 m minimum).

Checkpoint: Simple Look-Ahead Behaviour

Before implementing full Pure Pursuit, verify your path-following logic using a simpler look-ahead rule.

Replace your target selection with:

```
1 target = self.path[min(self.closest_idx + 5, len(self.path) - 1)]
```

This creates a basic look-ahead controller similar to Lab 7.

If the robot follows the path smoothly, your indexing and path generation are correct.

Then replace this with the full Pure Pursuit algorithm.

Implementation

Create `mission_runner.py`:

```
1 #!/usr/bin/env python3
2 """
3 Mission Runner - follows a multi-waypoint path using Pure Pursuit.
4 Designed for Ackermann-steered robots (minimum turning radius matters).
5
6 Subscribes: /tracert/gps (turtlesim/Pose from the God Node)
7 Publishes: /auto/cmd_vel (geometry_msgs/Twist)
8
9 The Farm Manager arbitrates /auto/cmd_vel into /cmd_vel, gated by the
10 keyboard spacebar deadman.
11 """
12 import math
13 import rclpy
14 from rclpy.node import Node
15 from geometry_msgs.msg import Twist
16 from turtlesim.msg import Pose
17
18 # --- TUNING ---
19 LOOK_AHEAD = 0.30 # Pure Pursuit look-ahead distance (m)
```



```

20 CRUISE_SPEED      = 0.12      # Forward speed on rows (m/s)
21 TURN_SPEED        = 0.08      # Forward speed inside headland turns (m/s)
22 MAX_YAW_RATE       = 1.0      # Safety cap on angular.z (rad/s)
23 ARRIVE_TOLERANCE   = 0.15     # Distance to last waypoint that counts as done (m)
24 TIGHT_CURVATURE    = 1.0      # Curvature threshold to switch to TURN_SPEED (1/m)
25
26 # --- HEADLAND GEOMETRY ---
27 # Your robot's min turning radius is ~0.35 m. Use 0.50 m for margin.
28 TURN_RADIUS        = 0.50
29
30
31 def clamp(x, lo, hi):
32     return max(lo, min(hi, x))
33
34
35 def normalize_angle(a):
36     return math.atan2(math.sin(a), math.cos(a))
37
38
39 def generate_zigzag_path():
40     """Build a two-row zigzag with a single-arc headland turn.
41
42     Returns a list of (x, y) breadcrumbs. Spacing is ~0.1 m, dense
43     enough for Pure Pursuit to always find a look-ahead point.
44     """
45     path = []
46     step = 0.1
47
48     # --- Row 1: west to east at y = -TURN_RADIUS ---
49     x_start, x_end = -1.0, 1.0
50     y_row1 = -TURN_RADIUS
51     n = int((x_end - x_start) / step)
52     for i in range(n):
53         alpha = i / n
54         x = x_start + (x_end - x_start) * alpha
55         path.append((x, y_row1))
56
57     # --- Headland arc: 180 deg, centred at (x_end, y_row1 + TURN_RADIUS) ---
58     # Same parametric-circle trick as Lab 7 Segment 2.
59     cx, cy = x_end, y_row1 + TURN_RADIUS
60     n_arc = 40
61     # Angle starts at -pi/2 (bottom of circle) and sweeps to +pi/2 (top)
62     for i in range(n_arc + 1):
63         theta = -math.pi / 2 + (math.pi * i / n_arc)
64         x = cx + TURN_RADIUS * math.cos(theta)
65         y = cy + TURN_RADIUS * math.sin(theta)
66         path.append((x, y))
67
68     # --- Row 2: east to west at y = +TURN_RADIUS ---
69     y_row2 = y_row1 + 2 * TURN_RADIUS
70     for i in range(n):
71         alpha = i / n
72         x = x_end - (x_end - x_start) * alpha
73         path.append((x, y_row2))
74
75     return path
76
77
78 class MissionRunner(Node):
79     def __init__(self):
80         super().__init__('mission_runner')
81
82         self.create_subscription(Pose, '/tractor/gps', self.on_pose, 10)
83         self.pub = self.create_publisher(Twist, '/auto/cmd_vel', 10)
84
85         self.pose = None
86         self.path = generate_zigzag_path()
87         self.closest_idx = 0
88         self.mission_complete = False
89         self.last_pose_time = self.get_clock().now()
90
91         self.create_timer(0.05, self.tick)    # 20 Hz
92         self.get_logger().info(

```

```

93         f"Mission Runner ONLINE. {len(self.path)} breadcrumbs loaded."
94     )
95
96     def on_pose(self, msg: Pose):
97         self.pose = msg
98         self.last_pose_time = self.get_clock().now()
99
100     # --- Pure Pursuit core ---
101     def find_lookahead_target(self):
102         """Find the first breadcrumb at least LOOK_AHEAD metres ahead.
103
104         Also advances self.closest_idx so we don't scan the whole path
105         every tick.
106         """
107         px, py = self.pose.x, self.pose.y
108
109         # Advance closest_idx while the next breadcrumb is still within
110         # look-ahead distance behind us.
111         while self.closest_idx + 1 < len(self.path):
112             nx, ny = self.path[self.closest_idx + 1]
113             if math.hypot(nx - px, ny - py) < LOOK_AHEAD:
114                 self.closest_idx += 1
115             else:
116                 break
117
118         # Walk forward from closest_idx looking for a point at least L_d ahead.
119         for i in range(self.closest_idx, len(self.path)):
120             tx, ty = self.path[i]
121             if math.hypot(tx - px, ty - py) >= LOOK_AHEAD:
122                 return (tx, ty, i)
123
124         # Fell off the end. Return the last point.
125         return (self.path[-1][0], self.path[-1][1], len(self.path) - 1)
126
127     def pure_pursuit_cmd(self, target):
128         """Compute (linear, angular) to drive toward the look-ahead point."""
129         tx, ty, _ = target
130         px, py, heading = self.pose.x, self.pose.y, self.pose.theta
131
132         # Angle from robot heading to the look-ahead point.
133         angle_to_target = math.atan2(ty - py, tx - px)
134         alpha = normalize_angle(angle_to_target - heading)
135
136         # Pure Pursuit curvature: kappa = 2 * sin(alpha) / L_d
137         curvature = 2.0 * math.sin(alpha) / LOOK_AHEAD
138
139         # Choose cruise vs turn speed based on how tight the curvature is.
140         speed = TURN_SPEED if abs(curvature) > TIGHT_CURVATURE else CRUISE_SPEED
141
142         # Yaw rate for Twist. Motor driver converts to steering angle.
143         yaw_rate = clamp(curvature * speed, -MAX_YAW_RATE, MAX_YAW_RATE)
144
145         return speed, yaw_rate
146
147     def tick(self):
148         cmd = Twist()
149
150         if self.pose is None:
151             self.pub.publish(cmd)
152             return
153
154         # GPS staleness check. If we haven't seen a pose update in 2 s,
155         # something is wrong, publish zero and warn.
156         age = (self.get_clock().now() - self.last_pose_time).nanoseconds / 1e9
157         if age > 2.0:
158             self.get_logger().warn(
159                 f"GPS stale ({age:.1f}s old). Holding zero.",
160                 throttle_duration_sec=2.0)
161             self.pub.publish(cmd)
162             return
163
164         if self.mission_complete:
165             self.pub.publish(cmd)

```

```

166         return
167
168     # Done? (close enough to the last breadcrumb)
169     last_x, last_y = self.path[-1]
170     if math.hypot(last_x - self.pose.x,
171                 last_y - self.pose.y) < ARRIVE_TOLERANCE:
172         self.get_logger().info("Mission complete. Holding.")
173         self.mission_complete = True
174         self.pub.publish(cmd)
175         return
176
177     # Pure Pursuit control.
178     target = self.find_lookahead_target()
179     linear, angular = self.pure_pursuit_cmd(target)
180
181     cmd.linear.x = linear
182     cmd.angular.z = angular
183     self.pub.publish(cmd)
184
185
186 def main():
187     rclpy.init()
188     node = MissionRunner()
189     try:
190         rclpy.spin(node)
191     except KeyboardInterrupt:
192         pass
193     finally:
194         # SAFETY: zero the auto command on shutdown.
195         stop = Twist()
196         node.pub.publish(stop)
197         node.destroy_node()
198         rclpy.shutdown()
199
200
201 if __name__ == '__main__':
202     main()

```

Debugging Hint: Path Skipping

If the robot appears to jump ahead or skip sections of the path:

- Your look-ahead distance may be too large
- Your breadcrumb spacing may be too coarse

Reduce the look-ahead distance or increase path resolution.

What's new vs. Lab 9

- **A breadcrumb path**, generated the same way as Lab 7 (linear interpolation plus parametric arcs).
- **Pure Pursuit** instead of naive “point at goal.” Handles curved paths and waypoints behind you gracefully.
- **Single-arc headland turn** using `TURN_RADIUS = 0.50 m`, well above your robot’s 0.35 m minimum.
- **Publishes to `/auto/cmd_vel`**, not `/cmd_vel`. The Farm Manager arbitrates.
- **Fixed-rate control loop**. 20 Hz, independent of how often the God Node publishes.
- **GPS staleness check**. If `/tractor/gps` stops updating, the runner stops publishing motion. Real systems must handle sensor dropouts.

Why not the Lab 9 P-controller?

Your Lab 9 controller computes `angular.z = Kp_ang * angle_error`. For an Ackermann vehicle, `angular.z` is yaw rate, and the motor driver converts it to a steering angle via $\delta \approx \arctan(L \cdot \omega / v)$.

When forward speed v is small, this math tries to produce huge steering angles. Pure Pursuit sidesteps this entirely: curvature is computed from geometry, and yaw rate is derived by multiplying curvature by the currently chosen speed. The steering angle command is always physically achievable.

Part 6: End-to-End Bench Test

Still on your laptop (robot on blocks is fine, or not running at all yet):

```
1 # Terminal 1: simulated GPS
2 ros2 run lab9 fake_god_node --ros-args -p mode:=static
3
4 # Terminal 2: arbiter (HOLD SPACEBAR IN THIS WINDOW or any focused window)
5 ros2 run lab10 farm_manager
6
7 # Terminal 3: mission runner
8 ros2 run lab10 mission_runner
9
10 # Terminal 4: observe the motor commands
11 ros2 topic echo /cmd_vel
```

Integration Test Checklist

1. Spacebar **released**: `/cmd_vel` is zero regardless of what the mission runner publishes.
2. Spacebar **held**: mission runner output reaches the motors. You should see non-zero `linear.x` and `angular.z` values.
3. Release the spacebar mid-mission: `/cmd_vel` snaps to zero, wheels stop.
4. Re-engage spacebar: mission resumes from the current path position.
5. Kill the Mission Runner with Ctrl+C while holding spacebar: `/cmd_vel` drops to zero (no auto source).

With the fake God Node in `static` mode, the simulated robot pose is fixed at (1.0, 0.5), so the mission runner will try to drive toward the first breadcrumb but the simulated pose won't change. You won't see it physically advance through the path. That requires a real pose feedback loop. For a dynamic test, use `mode:=drift` or wait for Part 8 (real ceiling camera).

Real-World Behaviour Note

The ceiling camera pose is not perfectly smooth, and you may observe small jitter in position, delayed updates, and occasional noise in heading. If the robot behaves poorly reduce the forward speed and increase the look-ahead distance. **Real robots always require tuning for imperfect sensors.**

Part 7: One-Click Launch

Create `lab10/launch/farm.launch.py`:

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4 def generate_launch_description():
5     return LaunchDescription([
6         Node(package='lab10', executable='farm_manager',
7             name='farm_manager',
8             output='screen'),
```

```

9     Node(package='lab10', executable='mission_runner',
10           name='mission_runner',
11           output='screen'),
12 ]))

```

Why output

The Farm Manager's deadman messages ("DEADMAN engaged", "DEADMAN released") only show up if you set `output='screen'`. Without that flag, `ros2 launch` hides node log output by default. You want to see the deadman state, so always set it for safety-critical nodes.

Register the launch folder in `setup.py`:

```

1 import os
2 from glob import glob
3
4 # inside setup():
5 data_files=[
6     ('share/ament_index/resource_index/packages',
7      ['resource/lab10']),
8     ('share/lab10', ['package.xml']),
9     (os.path.join('share', 'lab10', 'launch'),
10      glob('launch/*.py')),
11 ],

```

Rebuild and try it:

```

1 cd /workspaces/agtech_ros2
2 colcon build --packages-select lab10
3 source install/setup.bash
4 ros2 launch lab10 farm.launch.py

```

Two nodes, one command. You'll add the real God Node and weed detector here in the Final Project.

Part 8: Transplant to the Real Robot

Same procedure as Lab 9, but with one important difference: **the Farm Manager stays on your laptop**, not the robot. The keyboard deadman needs a keyboard to listen to, and the robot is headless.

Architecture:

- Your laptop: Farm Manager + Mission Runner + spacebar deadman.
- Instructor's laptop: God Node publishing `/tractor/gps`.
- Robot: motor driver listening for `/cmd_vel` over Wi-Fi.

Set `ROS_DOMAIN_ID` to match the rest of your group's setup. The Farm Manager's `/cmd_vel` bridges to the robot exactly the way Lab 9's controller did.

On-Robot Safety Protocol

1. Robot on blocks, wheels free.
2. Laptop: launch `farm.launch.py`.
3. Spacebar released: wheels must be still.
4. Hold spacebar: verify wheels respond to the Mission Runner's commands.
5. Release spacebar: wheels stop within 100 ms.
6. **Only after all of the above:** place the robot on the floor for a full mission test.

During the floor test, the instructor's ceiling camera provides real pose on `/tractor/gps`, and your robot should follow the zigzag path autonomously. The moment anything looks wrong, release the spacebar.

Stretch Goals (Optional)

1. **Tighter row spacing.** Modify `generate_zigzag_path` to put rows 0.5 m apart instead of 1.0 m. A single 0.5 m arc won't work (your robot's minimum is 0.35 m radius / 0.70 m diameter). Implement

a three-phase manoeuvre: drive past the row end, full steering in one direction, reverse with inverted steering, then forward into the next row.

2. **Mission from YAML.** Load waypoints from a parameter file rather than hardcoding them in `generate_zigzag_path`.
3. **Speed scheduling.** Slow down on curves, speed up on straights. You already have access to the path and the current index; compute the curvature of the next 5 breadcrumbs and modulate `CRUISE_SPEED`.
4. **Visual deadman indicator.** The terminal log is fine, but a coloured indicator in a small Tkinter window would be more visible during a live demo. Add a green/red light driven by deadman state.

Deliverables

Submit a PDF containing:

1. **Safety test evidence.** Screenshot or short video showing that releasing the spacebar zeros `/cmd_vel` while the mission runner is actively publishing to `/auto/cmd_vel`.
2. **Code.** Your `farm_manager.py` and `mission_runner.py`.
3. **Picture.** Your robot following the zigzag under the ceiling camera, OR on blocks reacting to `fake_god_node` in drift mode, with the spacebar deadman demonstrated at least once mid-mission.
4. **Reflection (5 sentences).** In the Farm Manager, the deadman check runs in a 20Hz timer, not directly inside the keyboard callbacks. Explain what could go wrong if it *only* ran inside the keyboard callbacks. Then explain why your Mission Runner also uses a timer-based control loop instead of reacting to every `/tractor/gps` message.
5. **Extra: Path plot.** A plot of the breadcrumb path generated by `generate_zigzag_path`. Any way is great, for example a Matplotlib in a separate script is fine.

What's Next: The AgTech Triathlon

You now have the full stack:

- **Lab 4:** Seed dispenser (actuation)
- **Lab 6:** Weed detection (perception)
- **Lab 7:** Breadcrumb path generation (planning)
- **Lab 9:** Ceiling camera GPS bridge (external state estimation)
- **Lab 10:** Ackermann mission execution plus human override (safety and control)

In the Final Project, the mission runner loads Lab 7's full field path, the God Node provides live pose, the weed detector triggers the seeder, and through all of it the Farm Manager lets a human take over the second anything looks wrong.

You're building a real agricultural robot. The control architecture here is the same stuff PtX Trimble uses. Same pattern, different scale.